Code Review Record: Reservation/reservation_service.py

Team: Vibranium
Date: Oct 21,2026
Time: 7:30 to 8:55
Code reviewed: Reservation/reservation_service.py
Why this file: highest complexity per line in the codebase, plus it holds the core booking rules: conflict detection, capacity checks, and weekly repeats.

Role	Name	Present?
Author (committer)	Sittipruk Puttisarn (Jan)	Yes
Reviewer	Su Pyae Pyae Aung (Micky)	Yes
Reviewer	Thiri Htet Thwe (Thiri)	Yes
Scribe / facilitator	Zin Zin Zaw Win (Zin Zin)	Yes

Author's overview

Jan: This file handles creating, editing, and deleting room reservations, plus printing out the weekly timetable. It has to make sure bookings do not overlap in the same room and that groups are not bigger than room capacity. The parts I was most unsure about were how weekly repeating schedules handle weird dates when the semester ends on a day that is not a class day, and how we should check room renovations without circular imports from Micky's RoomService.

Summary

- Issues found: 9 (Major: 2, Minor: 5, Questions: 2)
- Verdict: Accept / **Accept with fixes** / Re-review (circle one)
- Fixes due: Oct 22, 2026

Issues

Line numbers are from the reviewed commit.

ID	Lines	Severity	Category	Raised by	Issue (short)	Decision
CR-1	382–395	Major	Correctness	Thiri	Never checks if the room is closed	Accept: add availability check
CR-2	295–306	Major	Correctness	Micky	Weekly schedule shows "ghost" classes	Accept: check meeting day offset
CR-3	443–451	Minor	Correctness	Jan	ONCE bookings accept a junk end_date	Accept: disallow different end_date

CR-4	369–404	Question	Spec	Zin Zin	No semester date limits	Keep in gateway; document it
CR-5	215–222, 236–239, 254–258	Minor	Maintainability	Thiri	Same "not found" error 3 times	Accept: call get_reservation()
CR-6	331–354	Minor	Maintainability	Micky	Magic number 79 in the export	Accept: make REPORT_WIDTH constant
CR-7	12–27	Question	Dependencies	Jan	Two copies of the import list	Keep try/except for unit tests
CR-8	45–59	Minor	Linter (B042)	Zin Zin	Exception doesn't pass all args to super()	Leave it; don't break gateway str()

CR-1: Bookings ignore room closures (Major)

```
self._validate_fields(reservation)

room_capacity = self._room_service.get_room_capacity(
        reservation.room_id
)
...
existing = self._find_conflict(reservation)
```

Problem: we check fields, capacity and clashes, but never whether the room is marked unavailable (renovation and so on). That's a Must Have requirement, and test plan case 5.8 expects the booking to be rejected.

Suggested fix: add `is_room_available(room_id, start, end)` to `RoomCapacityProvider`. The Room service already has `is_room_available()`. Call it here with `start_date` and `last_date`, and raise a new `RoomUnavailableError`. The gateway adapter then implements the method and maps the error to an HTTP status.

Discussion: Thiri said the gateway already stores unavailable periods in SQLite, so forwarding the call is easy. Jan agreed this keeps ReservationService clean without needing to import RoomService directly.

Action / owner: Jan adds the check and `RoomUnavailableError` in `reservation_service.py`. Micky exposes availability through the Room service. Thiri adds `is_room_available()` to the gateway's `RoomCapacityLookup` and maps the new error to HTTP 409. Add a test for test plan case 5.8.

Fixed in: commit pending

CR-2: Weekly schedule shows "ghost" classes (Major)

```
week_end = week_start + timedelta(days=6)
 ...
 for res in candidates:
        if has_date_overlap(
        res.start_date, res.last_date, week_start, week_end
        ):
        matched.append(res)
```

Problem: this checks that the date ranges overlap, not that the class actually meets that week. Reproduced: a weekly Monday class from 5 Oct to 8 Oct only meets on Mon 5 Oct, but asking for the week starting Wed 7 Oct still lists it. The same thing happens with Monday-start weeks whenever `end_date` isn't itself a meeting day.

Suggested fix: a reservation meets at most once per week, so work out that one date and check it:

```
offset = (res.start_date.weekday() - week_start.weekday()) % 7
 meeting = week_start + timedelta(days=offset)
 if res.start_date <= meeting <= res.last_date:
        matched.append(res)
```

Add a test for the 5 - 8 Oct case.

Discussion: Micky showed the calendar dates on screen to prove the bug. Jan tested the math on scratch paper and confirmed this fixes it completely without needing any slow loops.

Action / owner: Jan replaces the overlap check in `get_schedule()` with the meeting-date check above, and adds a test for the 5–8 Oct case.

Fixed in: commit pending

CR-3: ONCE bookings accept a junk end_date (Minor)

```
if reservation.repetition is Repetition.WEEKLY:
        if reservation.end_date is None:
        ...
```

Problem: `end_date` is only validated for WEEKLY bookings. A ONCE booking with `end_date` = 1 Jan (before its start) was accepted. `last_date` ignores it so nothing breaks right now, but the stored data is misleading.

Suggested fix: for ONCE, reject (or set to `None`) any `end_date` that isn't `None` or equal to `start_date`.

Discussion: Jan admitted he just forgot an else: branch when writing the validation. We all agreed it is better to fail early if someone sends conflicting start/end dates for a single meeting.

Action / owner: Jan adds an `else:` branch in `_validate_fields()` that raises `InvalidReservationError("One-time reservations cannot have a different end date.")`, plus a test.

Fixed in: commit pending

CR-4: No semester date limits (Question)

Problem: nothing stops a booking in 2030 or in last year. Test plan case 5.11 expects "outside the semester" to be rejected.

Options: add `SEMESTER_START` and `SEMESTER_END` constants and check them in `_validate_reservation`, or drop case 5.11 from the test plan.

Team decision: We decided to keep semester dates out of ReservationService. Semester dates change every year, and hardcoding them in the core service would break past test data and make it annoying to test future semesters. The API Gateway and Zin Zin's frontend forms already validate against the current semester (Aug 17 - Dec 14, 2026), which is the right place for it.

CR-5: Same "not found" error in three places (Minor)

```
raise ReservationNotFoundError(
        f"Reservation with ID '{reservation_id}' not found."
)
```

Problem: this check and message are copy-pasted in `edit_reservation`, `delete_reservation` and `get_reservation`.

Suggested fix: in `edit_reservation`, call `self.get_reservation(reservation_id)`, which already raises the error, so the message lives in one place.

Discussion: Thiri noticed this while reading the CRUD methods. It is only 3 lines of duplication, but reusing get_reservation() makes the code cleaner.

Action / owner: Jan replaces the check in `edit_reservation()` with `self.get_reservation(reservation_id)`; the existing edit tests cover it.

Fixed in: commit pending

CR-6: Magic numbers in the text export (Minor)

```
"=" * 79,
 ...
f"{r.competency:<18} {r.group_size:<10} "
```

Problem: 79 appears four times, along with the widths 10, 15 and 18. A competency name longer than 18 characters pushes the columns out of line.

Suggested fix: add `REPORT_WIDTH = 79` at the top, and cut long names with `f"{r.competency[:18]:<18}"`.

Discussion: Micky pointed out that CMKL competency codes are often long. Jan said truncating to 18 characters in the plain-text export is fine since the full name is always visible in the web UI anyway.

Action / owner: Jan adds `REPORT_WIDTH = 79` next to `ID_LENGTH`, uses it in all four places, and cuts long competency names.

Fixed in: commit pending

CR-7: Two copies of the import list (Question)

```
try:
        from .reservation_data import (...)
except ImportError:
        from reservation_data import (...)
```

Problem: the same five names are imported twice, once as a package and once as a plain module. If the list changes, both copies have to change.

Options: agree on one way to run the code (the gateway already adds `Reservation/` to `sys.path`) and keep a single import block.

Team decision: We decided to keep the try/except block. It is a bit redundant, but it lets us run tests directly inside the folder (python test_reservation_service.py) and also through unittest discovery from the root (python -m unittest discover). It saves us headaches when running tests quickly.

CR-8: Linter warning B042 on ReservationConflictError (Minor)

Problem: `super().__init__(message)` drops `conflicting_reservation`, so `copy` and `pickle` can't rebuild the error.

Note: the obvious fix, passing both arguments to `super()`, turns `str(err)` into a tuple and breaks the gateway's error message. Leaving it and recording the reason is acceptable.

Team decision: Leave it as-is. We do not pickle exceptions anywhere in our app, and keeping str(err) clean for API error messages in the gateway is way more important. Jan will just add a comment explaining why.

Strengths noted

- **lines 128–155:** `is_conflicting()` is short and correct. Two reservations on the same weekday whose date ranges overlap always share a real meeting date (the later start date is one), so there's no need to loop week by week.
- **lines 176–224, 369–451:** validation is cleanly layered: field checks, then room checks, then conflicts, shared by create and edit.
- **lines 418–421:** `_find_conflict()` skips the reservation's own saved copy, which is what makes "move my class an hour later" edits work (test plan 6.1).
- **lines 33-5**0: Having specific custom typed exceptions (CapacityExceededError, ReservationConflictError, etc.) made it really easy for Thiri to map each error to the right HTTP status code in the API Gateway.

Follow-up (filled in after the fixes)

ID	Planned fix	Owner	Fixed?	Commit
CR-1	check room availability before booking; new `RoomUnavailableError` (409)	Jan, Micky, Thiri	Yes	pending
CR-2	`get_schedule()` checks the actual meeting date in that week	Jan	Yes	pending
CR-3	ONCE bookings reject a different `end_date`	Jan	Yes	pending

CR-4	team decision: semester limits, or drop test 5.11	Team	Yes	pending
CR-5	edit_reservation() reuses get_reservation()	Jan	Yes	pending
CR-6	REPORT_WIDTH constant; cut long names	Jan	Yes	pending
CR-7	team decision: one import style	Team	Yes	pending
CR-8	leave it, with the reason recorded	Jan	Yes	pending

Scribe: Zin Zin Zaw Win, signed off Oct 21, 2026

Team Review Reflection

1. What we found (bugs, good things, etc.)

When we reviewed reservation_service.py together, we found some really solid logic and two important bugs. On the positive side, Jan's time overlap math is really smart. Using half-open intervals [start, end) means back-to-back classes (like 10:00-12:00 and 12:00-14:00) do not block each other, and checking weekday plus date overlap avoids having to write messy loops over every week of the semester. The custom exceptions also made it really easy for the gateway to return clear error messages to the frontend.
For bugs, the biggest one was CR-1: the service checked room capacity and existing reservations, but completely forgot to check if the room was closed for renovations or maintenance. This was a Must Have requirement in test case 5.8. The other big bug was CR-2 in get_schedule(), where a weekly class could show up as a 'ghost' class in a week where it did not actually meet, just because the overall start and end dates overlapped. We also caught smaller issues like copy-pasted error messages, magic numbers in the timetable export, and ONCE bookings quietly accepting invalid end dates.

2. What changes we made as a result

After talking through all the issues, Jan agreed to make the fixes. For CR-1, we decided to extend RoomCapacityProvider with is_room_available() so the reservation service can check room status through the existing interface without adding messy circular imports. For CR-2, Jan replaced the simple date range check in get_schedule() with a formula that calculates the exact meeting weekday offset for that week:

```
offset = (res.start_date.weekday() - week_start.weekday()) % 7
meeting = week_start + timedelta(days=offset)
if res.start_date <= meeting <= res.last_date: matched.append(res)
```

For the smaller cleanups, we added validation to disallow different end dates on ONCE bookings, reused get_reservation() inside edit_reservation() so the error message only lives in one place, defined REPORT_WIDTH = 79 as a constant, and cut competency titles at 18 characters so long names do not wreck the terminal timetable formatting.

3. What worked well in our review process

Assigning clear roles beforehand made the meeting run smoothly. Jan walked us through the logic as the author, Micky and Thiri questioned edge cases as reviewers, and Zin Zin took notes as the scribe without anyone having to stop talking. Because everyone had a specific job, nobody talked over each other and the session felt like teamwork rather than criticism.
Another thing that worked really well was having the terminal open with our test suite ready. Whenever someone asked a 'what if' question-like whether a one-off class on a Wednesday would clash with a

Monday weekly booking-we ran the tests right away or wrote a quick 3-line test case to verify the behavior. That saved us from wasting time arguing about hypotheticals because we could just run the code and see the result immediately.

4. What didn't work well

The main thing that did not go well was time management in the first half of the hour. We got stuck discussing has_time_overlap and has_date_overlap for almost 25 minutes. Even though those functions are only a few lines long, we kept re-checking whether <= or < was right for dates and times. It turned out the logic was already correct, so we ended up spending almost half our time on code that did not even have a bug.

Because of that early time sink, we had to rush through the second half of the file. We barely spent 10 minutes on get_schedule() and export_schedule_text(), which is actually where CR-2 and the formatting issues were hiding. If we had paced ourselves better from the start, we would have had more time to look into the export formatting and search functions.

5. What we will do differently next time

Next time, we will set a strict timer for each section of the file before we start. For example, give ourselves 10 minutes for data structures, 15 minutes for conflict algorithms, 20 minutes for validation and CRUD, and 15 minutes for wrap-up. The scribe can warn us when time is up for a section so we do not get bogged down on the first few methods.

We also agreed that for our next code review, everyone should read the file individually before the meeting and write down their questions. Coming into the call with our notes already prepared would have let us jump straight to the suspicious parts (like room closures and ghost classes) instead of spending 20 minutes reading simple helper functions out loud together.